

Everything the shop did today, already added up.

A billing, stock and double-entry accounting application for Ugandan shops — a till that works at counter speed, books that balance without anybody keeping them, and a printed receipt at the end of it. This is what is inside: the layers, the modules, the data, the screens, the design system, and the sixteen rounds of correction that made each of them what they are.

48,754 lines of application code 29 backend modules 325 endpoints 73 tables 31 screens
58 reports

Contents

01 The concept	10 Screens
02 Shape & scale	11 Workflows
03 The layers	12 The design system
04 Storage	13 Paper
05 Companies	14 Verification
06 Identity	15 How it got here
07 Modules	16 Known gaps
08 The data model	17 Running it
09 Money & tax	

One book, kept by the act of selling

The premise is that a shopkeeper should never keep books. They should sell, buy, and take money — and the books should be a by-product of having done so, complete enough to file tax from and to argue with a supplier over.

Every till sale posts, in one transaction: the invoice and its lines, the stock movement out of the right batch, the journal entries that debit cash and credit sales and move cost of goods, and the party balance if it was on credit. There is no “post to accounts” step, because a step somebody can forget is a set of books that is wrong by the amount they forgot.

Three commitments shape almost every decision below.

- **It keeps working when the network does not.** The desktop build is the whole application — server, database and interface — on one machine. Nothing about a sale requires a connection.
- **Saved means on the platter.** A write the till has been told succeeded is on disk before the reply leaves the socket, not queued behind an event loop turn.
- **Nothing silent.** A figure that could not be read shows as unavailable, never as `Sh 0`. A backup that cannot be re-opened is deleted rather than left in the list looking usable.

A stale figure and a suppressed one are different, and only one of them is honest.

THE RULE THAT SURVIVED EVERY ROUND

What the thing actually weighs

22,021 lines of backend JavaScript	26,733 lines of React	6,706 lines of CSS	325 HTTP endpoints
73 database tables	124 settings	51 permission questions	58 reports

The endpoint mix is telling: **192 reads to 133 writes**. This is an application that is looked at far more than it is typed into — which is why the reports, the dashboard and the panels get as much attention as the till.

PIECE	WHAT IT IS	SIZE
backend/	Express, 29 modules, 39 route files, 5 services, shared engines	22,021
frontend/src/	React 18 + Vite, 31 pages, 18 shared library modules	26,733
deck.css	the live design layer — tokens, components, the type and spacing scales	4,908
styles.css	the original layer: reset and base rules, 148 live selectors	1,798
electron/	desktop shell — the same server, the same bundle, one window	2 files
test/ + e2e/	8 unit suites, 4 API suites, 20 browser specs, 5 audits	37 files

What a request passes through

No framework magic and no ORM. A request crosses six layers, each of which can be read in one sitting, and every one of them has a rule it exists to hold.

express
server.js

Helmet, compression, CORS, request logging, a 50 MB JSON limit, and one envelope for every reply — `res.success(data, message)` / `res.fail(message, status)`. The client never has to guess the shape.

auth middleware/auth.js	Verifies the token, then <i>asks the database whether it still means anything</i> : the session is not revoked, the company is not suspended, the membership still stands. Each failure names which. Then it puts the company into storage context.
routes modules/*/...routes.js	One file per domain concern. Permission is checked per route against the role's granted pairs; validation refuses in the shopkeeper's language, not the schema's.
durable shared/durable.js	The transaction idiom, written once: acquire, begin, work, commit, reply. The handler returns a <i>function</i> that sends the response — so the client is never told “saved” before the bytes are on the platter. Return anything else and it rolls back.
engines shared/*.js	The parts with opinions: the tax chain, the double-entry poster, the stock poster with earliest-expiry-first consumption, unit conversion, sequences, the provisioner that builds a whole company.
storage database/db.js	sql.js — SQLite compiled to WebAssembly. One store per database file, each with its own handle, write mutex, transaction depth and atomic save.

Why sql.js and not a native driver

No native compilation means the same artifact runs in Electron on a shop's laptop, on a Linux host, and in a restricted CI sandbox — no build toolchain at the counter. The cost is that the whole database lives in a 2 GB WebAssembly heap and every committed write exports the entire file. Both consequences are measured rather than assumed; see [Storage](#).

04 STORAGE

A ceiling that is known, and said out loud

The storage layer is the most carefully argued file in the application, because it is the one where being wrong loses a sale that has already been rung up.

INVOICES	ROWS	FILE	OPENS IN	WORST ENDPOINT
20,000	380k	31.8 MB	0.6 s	1.3 s

INVOICES	ROWS	FILE	OPENS IN	WORST ENDPOINT
100,000	1.9M	169 MB	1.9 s	2.2 s
300,000	5.6M	486 MB	12 s	5.7 s
500,000	9.3M	989 MB	20 s	11 s

In trading terms 500,000 invoices is about **45 years at 30 sales a day**. The application warns at 600 MB — roughly 27 years — because “archive the oldest years” should be a calm decision, not an emergency. Past about a gigabyte the open fails with a WebAssembly trap that tells a shopkeeper nothing, so the layer catches it and says what it actually means.

THE FOUR PROPERTIES THAT TOOK THIRTEEN ROUNDS TO GET RIGHT

- **Atomic save.** Write a temp file, `fsync`, rename. A power cut can lose the last write; it cannot corrupt the file.
- **Flush at the end of the turn, commit synchronously.** An ordinary write is saved one event-loop turn later; a transaction saves before `commit()` returns. The earlier 200 ms debounce lost an acknowledged held bill to a `SIGKILL` 100 ms later — measured, not theorised.
- **Never export inside a transaction.** `export()` closes the handle, which discards the open transaction. The save defers instead.
- **One writer per file.** A mutex with a 30-second watchdog that complains loudly and takes the lock back, because a handler that never releases would wedge every till in the shop.

A BUG WORTH REMEMBERING

`Database.exec()` in `sql.js` leaks 16 bytes of WebAssembly stack per call and never restores the pointer. The old `query()` used it after every write to read the last insert id — so the process had a hard budget of about **328,000 writes** and then died with a memory trap, whatever the database contained. That was the ceiling the scale work kept hitting, and it was never a data-volume limit. `prepare()/step()/free()` has no such leak, verified to a million calls.

05 COMPANIES

One installation, many businesses

`firms` is the tenant boundary, and `firm_id` is threaded through 58 tables. A shopkeeper with a hardware shop and a bar runs both from one account, switches between them, and never sees one from inside the other.

Two storage arrangements, one application

The desktop build keeps everything in one file. Hosting cannot: sharing a file between shops makes every shop's sale export every other shop's data, turns a 45-year ceiling into about three months across two hundred shops, and makes one shop's till queue behind another's. So `SPLIT_STORAGE=1` gives each company its own SQLite file with a small central one for identity.

LIVES IN THE COMPANY'S FILE	LIVES IN THE CENTRAL FILE
Documents, stock, parties, the ledger, settings, roles and permissions, invoice numbering, the audit log — and counter staff , who belong to the shop they work in	The list of companies, accounts, memberships, invitations, sessions, verification codes, the sign-in throttle, the outbox

Which side `users` falls on was the decision the whole split turned on. A scan of every SQL statement in the codebase found **36 that straddled the boundary — and 33 of them were the same shape**: a company's rows joined to `users` to print who did it. Cashier on a receipt, who held a bill, who voided a line, the whole sales-rep half of Reports. Moving one table moved all of them at once, and took the count to zero.

A statement that names tables from both sides now **throws by name**. It has to: a `LEFT JOIN` across the boundary does not error, it returns the other side blank, and the screen renders a missing cashier as “Unknown” rather than as broken. A test walks every SQL literal in the tree and fails the build on a new one.

There is no distributed commit and none is invented. A write to the other database from inside an open transaction is refused, and the few operations that genuinely touch both — creating a company, accepting an invitation, restoring — order their writes so a crash between them leaves something repairable rather than something quietly wrong.

Backup, per company

A company exports as a real SQLite file carrying **control totals** — invoice count and value, purchases, parties, items, both ledger columns, stock valuation. The restore recomputes them inside the transaction and **rolls back on any mismatch**, naming which figure was wrong. Restoring in place needs no renumbering: ids are `AUTOINCREMENT` and unique across the file, so deleting a company's rows and reinserting them with their original ids leaves every foreign reference valid.

06 IDENTITY

The owner has an email.
The cashier does not.

Requiring an address per counter staff member means the owner invents `shop1cashier2@gmail.com`, or — worse and commonly — shares one login between two counters, which corrupts held bills. So there are two tiers.

TIER	WHO	SIGNS IN WITH	SCOPE
Account	Owner, admin, accountant	Email + password, verified by a six-digit code	Global — one account, many companies
Staff	Cashier, storekeeper	Username + password, or a 4-digit PIN	One company, named by its shop code

The shop code (`kampala-hardware`) is how a cashier says which shop before typing a username — because on a hosted server two shops may each employ a James. It is the name of the door, not the key: the sign-in screen may say a code was not recognised, but the route that accepts a password answers *“Wrong username or password”* whether the shop, the user or the password was wrong. A single-shop installation is never asked.

PERMISSIONS

Roles are per company and hold module/action pairs. The catalogue phrases all **51** of them as plain questions — “Give somebody access”, “See the all-businesses panel” — because an administrator ticking boxes should be answering questions about their shop, not reading an access-control matrix. A company's **owner** is not a role and cannot be edited out of their own books.

ONBOARDING

Sign up → a six-digit code (not a link: on Android the email frequently opens on a laptop) → the first business → invite staff. Sign-up answers identically whether the address is new or known, so it cannot be used to test which shopkeepers use the product; the person who holds the address is told by email that somebody tried. Invitations are 32 random bytes stored as a SHA-256 digest, single-use, seven days, withdrawable.

07 MODULES

Twenty-nine domains, one file each

sales Invoices, credit and cash, sales reps, idempotent posting	pos The till: keyboard billing, held bills, per-counter claims, day close
purchases Supplier bills, purchase orders, landed extra costs, batches	returns Credit notes, debit notes, delivery challans

estimates Quotations and sale orders, converted in one step	items Products, services, units, categories, barcodes, serials, repacks
parties Customers and suppliers, groups, per-party rates, statements	payments Receipts and payments, allocation against documents
money Cash and bank, movements, transfers, other income	expenses Categorised spending, straight into the ledger
accounting Chart of accounts, journals, trial balance, P&L, balance sheet, integrity	tax VAT, withholding, EFRIS register, filing history, a pre-filing checklist
reports 58 reports in 7 groups, each printable	dashboard Today, collection health, ageing, what needs attention
companies Create, switch, suspend, delete; people, invitations, the all-businesses panel	auth Three sign-in paths, sessions, refresh, shop resolution
users Staff logins, roles, permission grants	staff Who is on, what they sold, what they are owed
shifts Open and close the till, reconcile the drawer	stocktake Count the shelf, post the variance
manufacturing Recipes, production runs, component consumption, unit cost	offers Discount rules applied at billing time
loyalty Points earned and redeemed, per party	warranty Cover on what was sold, and claims against it
installments Payment plans and their schedules	recurring Scheduled sales and purchases, generated on a clock

settings 124 keys, tax rules, price lists, units, categories	system Backups, off-site copies, restore, updates, version
utilities Import, export, data checks and repairs	

08 THE DATA MODEL

Seventy-three tables, four families

DOCUMENTS

Sale invoices, sale returns, purchase invoices, purchase returns, purchase orders, estimates, challans — each a header and its lines, each carrying `firm_id` and a number from a per-company sequence.

STOCK

Items, `item_stock` (one row per batch), movements, barcodes, serials, repacks, stock takes. Batches carry an expiry; consumption is earliest-expiry-first with the no-expiry batches last.

BOOKS

Chart of accounts, journal entries and lines, account balances. Every document that moves value posts here in the same transaction that writes it.

COMPANY & IDENTITY

Firms, settings, roles and permissions, price lists, tax rules — plus the central accounts, memberships, invitations, sessions and auth codes.

THE SCHEMA DECLARES NO FOREIGN KEYS

Not one `REFERENCES` clause exists, so `PRAGMA foreign_keys = ON` — which the storage layer carefully re-applies after every save — has never enforced anything. Declaring them now would mean a table rebuild per constraint, and any orphan already in a shop's data would start hard-failing writes at the till. So the check lives in the test suite instead: **18 relationships, walked on the real database**, and it reports when it had nothing to check rather than passing silently. The shipped database has zero orphans — proved by planting two and watching the check name both.

A chain, not a slab

Tax is not a per-line rate. It is a firm-level chain of rules applied in order to the running total, each either *added* on top or *deducted* out of it. The Uganda default is two deductions:

1,000 → withholding tax 6% deducted (60) → 940 → VAT 18% deducted (169.20) → **770.80**

Rules can be scoped by document type and by how the sale was paid, so a cash sale at the till and a credit invoice can carry different treatment without anybody re-typing anything. Invoice-level rounding lets a shop round cash totals to the nearest 50 or 100 shillings, because coins below that are not really in circulation.

PRICES THAT ALREADY INCLUDE TAX

A shop that prices its shelf at 11,800 including 18% VAT is stating a total, not a base. With the setting on, the added taxes come back out first: sub-total 10,000, VAT 1,800, customer pays 11,800 — instead of 11,800 becoming 13,924. It applies only to *added* rules, because a deduction is taken out of the price the customer pays and there is nothing to include. Every line is rescaled, not just the total, so a line-level report still adds up to the invoice it came from.

THE MONEY COLOURS

Three semantic colours, used only for what they mean, and never as the only signal — roughly one man in twelve cannot separate the green from the red, and he is disproportionately likely to be the one behind the counter.

 received / in  owed to you  overdue  the one accent

Receivables ageing uses a four-step sequential ramp whose luminance is strictly decreasing in both themes, measured rather than assumed — and because the two palest steps sit under 3:1 against white, **every ageing step carries its figure beside it** rather than relying on the swatch.

10 SCREENS

A rail in four groups

Thirty-one screens, reached from an indigo rail organised by what a shopkeeper is trying to do rather than by which module owns the data. Every entry is gated twice — by permission and by a module switch — so a feature a shop has turned off is not

merely hidden, it is unreachable.

GROUP	SCREENS
Sell	Dashboard · Till · Sales · Parties
Stock	Items · Purchases · Stock take
Money	Cash & bank · Accounting · Reports
More	Staff · Shifts · Tax & URA · Data tools · Companies · Settings · Manufacturing · Offers · Warranty

The four that carry the work

- **Till** — keyboard-first: F1 focuses the scanner, Enter adds the top match, F2 discount, F8 hold, Ctrl+P saves and prints. Held bills are claimed per counter, so two tills signed in as the same user cannot resume the same bill.
- **Sales** — invoices, estimates, credit notes, challans, recurring schedules, instalments and loyalty behind one screen and one item table.
- **Reports** — 58 panes across Sales, Purchase, Money, Receivables, Payables, Stock and Tax & books. All 58 open and all 58 print.
- **Settings** — 9 pages after a regrouping that moved item and party defaults onto the screens they govern, with signposts left behind.

Shared components, not copies

The library is the antidote to the review's first finding — nineteen row menus in twelve files across two implementations, and five separate item tables. `doclines`, `docshell`, `rowmenu`, `moneyaccounts`, `reportshell`, `settingsui` and `printprompt` are each one implementation used everywhere. A component built twice is fixed once.

11 WORKFLOWS

A sale, end to end

- **Scan or search**
The item is found by barcode, name or code. Its price comes from the party's price list if one applies, then any offer rule that matches.
- **The total is computed twice**
Once in the browser so the figure moves as you type, once on the server on save. The client mirrors the engine exactly; the server is what decides.

- **One transaction**
Invoice and lines, stock out of the earliest-expiring batch, journal entries for cash or receivable, sales and cost of goods, the party balance, the audit row. All of it, or none of it.
- **Committed, then answered**
The commit writes the file synchronously. Only then does the reply go out — so “saved” cannot outrun the disk.
- **Print, if the shop wants it**
A dialogue offers the receipt, or prints straight through, or never asks — a setting with a per-save override. A failed print never makes a saved sale look unsaved.

Other paths worth knowing

- **Idempotent posting.** Every till sale carries a client-generated reference. A retry after a timeout returns the original invoice instead of ringing it up twice.
- **Held bills.** Parked at the counter, claimed per device, and offered back after a reload — flagged when the bill belongs to somebody else's session.
- **Day close.** Counts the drawer, reconciles against what the till recorded, and posts the variance.
- **Purchase → stock → cost.** A bill with extra landed costs spreads them across the lines, so the cost of goods on the eventual sale is the real one.

12 THE DESIGN SYSTEM

Two scales, one accent, and a rule about boxes

An outside design review found three visual layers coexisting and no system at all. What exists now is small enough to hold in your head, and — this is the part that matters — it is **measured by a tool that runs in the browser**, so the next change is judged by a number rather than by whether the screenshot feels tidier.

THE TYPE SCALE – EIGHT STEPS, EACH SIZE PAIRED WITH ONE WEIGHT

STEP	ROLE	WHERE
11.5 / 700 caps	caption	labels above fields, table headings
11.5 / 500	hint	the quiet second line under a figure
12.5 / 650	label	a chip, a pill, a small named thing
12.5 / 500	secondary	sub-headings, counts, notes

STEP	ROLE	WHERE
13.5 / 400	body	everything you read
13.5 / 650	title	card headings, tabs, buttons, rail labels
16 / 700	amount	money in a row or a total block
19 / 750	figure	the number a tile exists to show, and the page title

Before that pass the stylesheets held **25 distinct font sizes and 9 weights** — 12 and 12.5 and 13 and 13.5 all in play, 600 beside 650 beside 780 beside 900. Per page, the combinations roughly halved: Dashboard 20 → 13, Items 21 → 11, Cash & bank 18 → 9, Sales 15 → 8.

THREE COMBINATIONS NOBODY HAD CHOSEN

`<b>` inherits `bold` from the browser's own stylesheet, which resolves *relative to the parent* — so the same `<b>` came out 700 inside a 400 container and **900** inside a 650 one. No amount of reading the CSS finds that, because the weight is not in the CSS.

THE SPACING SCALE — A 2PX GRID TO 24, THEN COARSER

1 · 2 4 6 8 10 12 · 14 16 18 20 · 22 24 · 28 32 40 48 56 · 80

43 distinct values became 19, and **nothing moved by more than 2px** — which is precisely why it was safe. Nobody had chosen 9 over 8, or 13 over 12.

THE RULES THAT ARE NOT ABOUT NUMBERS

- **Two container levels, maximum.** A page section, and rows within it. Cards inside cards inside cards cost 32–48px of horizontal space per level and add a border the eye must parse. Audited: no page exceeds two.
- **One accent, one destructive, three money colours. Nothing else.** Audited: exactly two filled-button colours on every page.
- **Every money figure is tabular and right-aligned.** Columns that do not align at the decimal point cannot be scanned, and scanning them is the whole job. Audited: zero violations.
- **No new class name in a page file.** If a page needs a visual, it goes in the library.
- **Full screen for a document, a dialog for a decision** — and nothing sticky over scrolling content.

Row heights run 45px to 68px across pages, which looks like drift. It is not: every table uses the same cell padding and the same line height, and the heights differ because a party row carries a monogram, an item row a thumbnail, a money row a second line. Reports — a grid of numbers and nothing else — is already at the 45px the review asked for. Tightening the padding to make the numbers agree would have taken 4px off rows that need it and called the result density.

13 PAPER

The one surface nobody sees fail until a customer is holding it

Two formats out of one code path: an **A4/A5 document** for invoices and reports, and a **thermal receipt** for the counter, at 58mm or 80mm. Both are built as HTML and handed to the browser's own print pipeline — no PDF library, no server-side rendering, and therefore nothing that can be out of step with what is on screen.

- Every one of the 58 reports prints, which had not been true: the original page's table rules were overriding the print stylesheet and shredding the layout.
- The thermal receipt overflowed a real 3-inch roll by 14px, because a 76mm width was carrying 7px of padding on each side. `box-sizing: border-box` and a `max-width` fixed it — found by rendering it at the roll's real pixel width and looking.
- A shop can show a preview before printing or print straight through; the till always prints straight through, because a preview at the counter is a queue.

14 VERIFICATION

What has to be green before anything ships

LAYER	WHAT IT PROVES	COUNT
Unit test/*.test.js	The maths: tax chain, costing, units, tenant classification, referential integrity, the storage boundary	73
API test/*.e2e.js	Onboarding, staff sign-in, the storage split, the settings that used to do nothing	4 suites
Browser e2e/specs	Till, receipts, reports, concurrency, durability, backup, access, layout	20 specs
Audits e2e/*.mjs	Type and container discipline, unreachable controls, dead CSS, stylesheet influence	5 tools

LAYER	WHAT IT PROVES	COUNT
Static	Build, import graph, layer boundaries, <code>node --check</code> on every backend file	4 gates

Two checks deserve naming. **Layout** walks ten pages at four viewports and asserts every control is reachable and nothing scrolls sideways — the detector stops at the first ancestor that *scrolls*, because an earlier version stopped at the first `overflow: hidden` and reported 930 phantom failures. And the **shipped database is byte-identical** after every session: `md5 3485fb02...`, checked each time, so no amount of testing can quietly edit the thing that ships.

When a new test passes first time, assume it is broken until you have made it fail on purpose.

LEARNED TEN TIMES, THE HARD WAY

That rule is not a slogan. Ten checks in this project have passed while being incapable of failing: a guess counter rolled back by the transaction it was written in, so a six-digit code could be walked through all million values; an isolation test run as the account that owned both companies; a “both boxes are gone” assertion on a form that had never opened; control totals of zero matching zero. Each was found by trying to break the check, not the code.

15 HOW IT GOT HERE

Sixteen rounds of correction, then ten of construction

The numbering is real: each round or session is a commit, a report, and a set of checks that were green before it was called done.

R1–R6

Correctness, interaction, clipping, copy

Data bugs, layout that hid controls at real window sizes, and the discovery that settings were never applied in a first session.

R7–R8

A git history, and the design decisions

The work stopped being a pile of edits. Period controls, row menus and money semantics were settled once.

R9	Tenant isolation Five real cross-firm vulnerabilities, found by attacking the installation over HTTP as a second tenant.
R10–R11	Concurrency and durability The write mutex, the <code>exec()</code> stack leak, and an acknowledged write that a SIGKILL 100ms later could still lose.
R12–R13	Backup, restore, off-site A restore that reported success while deleting another firm's rows. Verification became part of the feature.
R14–R16	The screens, in detail The photographed screens, the sticky header reconsidered, and money that says what it settles and where it landed.
S1	Print on every save One prompt, three rules: only after a save succeeds; a failed print never unsaves; an explicit print instruction is not a question.
S2	Bulk update Four groups, row edits plus a bulk-actions bar, one audit row per changed field with old → new.
S3	Settings, regrouped 14 pages → 9, the sticky banner gone — and 25 switches found that no code anywhere read.
S4–S6	Many businesses Companies, per-company backup with control totals, and the all-businesses panel — every chart a sorted single-hue bar with its name written on it.
S7	The door Sign-up, verification, invitations, password reset — and a refresh token that had been walking past every revocation for thirty days.
S4.5–S4.6	One file per company The storage boundary, made real and enforced; then the shop code that lets a cashier say which shop before typing a username.

S8 **The settings that did nothing**

Five of the 25 built; one relabelled first, because “prices include tax” cannot mean that in a deduct chain.

S9–S10 **Type and spacing**

25 sizes → 9, 43 spacing values → 19, 60 dead CSS rules removed — and a measured correction to the review's advice to delete styles.css.

16 KNOWN GAPS

What is not true yet

A list that is kept honest is worth more than a list that is short.

GAP	WHY IT MATTERS	STATE
No mail provider chosen	Verification codes print to the server log until Postmark, Resend or SES is configured — with SPF, DKIM and DMARC on a sending domain	decision
20 settings still inert	Hidden from the Settings page rather than pretending, but still listed in the schema	mapped
styles.css not merged	148 live selectors to move into the design layer; 532 more to confirm by opening the screens that use them	measured
Per-company scheduled backups	Manual export and the forced copy on delete both work; the automatic per-company schedule does not exist	missing
Import as a new company	The renumbering path — “move this shop to a new laptop”. A deleted company's forced copy is not restorable through the interface	missing
Per-account rate limits	Reports and the all-businesses panel are the cheapest denial of service in the app	open
Offline is read-only by design	Selling requires a connection once hosted; every cached figure must say when it was last true	decided
Multi-currency	Single currency across businesses; the panel returns <code>currency_note: "single"</code> so the day that changes, the screen has somewhere to say so	decided
Billing	Plan, status, limits and trial dates carried from the first migration; nothing reads them and no provider is chosen	deferred

Three ways, one artifact

- **Desktop.** Electron wraps the same server and the same built bundle in one window. The database is a file in the application's data folder.
- **Server, one shop.** `node backend/server.js` with the frontend built; the backend serves `frontend/dist`, so there is one origin and no CORS in production.
- **Server, many shops.** The same, plus `SPLIT_STORAGE=1` and a central database — each company gets its own file, opened on demand and closed when idle.

SETTING	DOES WHAT
JWT_SECRET	Signs every session. Production refuses to start without it — a working default would let anyone who unzipped the installer forge an admin token
VYAPAR_DB_PATH	Where the database lives; the central one when storage is split
SPLIT_STORAGE	<code>1</code> for one SQLite file per company
TENANT_MAX_OPEN TENANT_IDLE_MINUTES	How many company files may be open at once, and how long an idle one is kept
EMAIL_PROVIDER EMAIL_FROM	<code>console</code> , <code>postmark</code> , <code>resend</code> , <code>smtp</code> or <code>none</code> , and the address they come from
APP_URL	The base for invitation links. Unset, links are built from the address the request arrived on — which is right more often than the development default
LOGIN_MAX_PER_IP LOGIN_MAX_PER_ACCOUNT	Sign-in throttle, now counted in the database so several processes share one set of attempts